

Aula 06: Superescalar

ACH2055 – Organização e Arquitetura de Computadores II

Arquitetura de Computadores: uma abordagem quantitativa. Patterson e Hennessy, 6ª edição – Capítulo 3
Arquitetura e Organização de Computadores. William Stallings, 8ª edição – Capítulo 14.

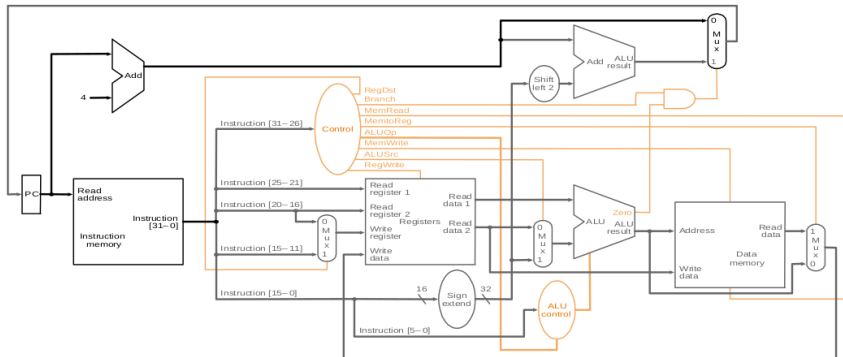
Valdinei Freire da Silva

Escola de Artes, Ciências e Humanidades - USP

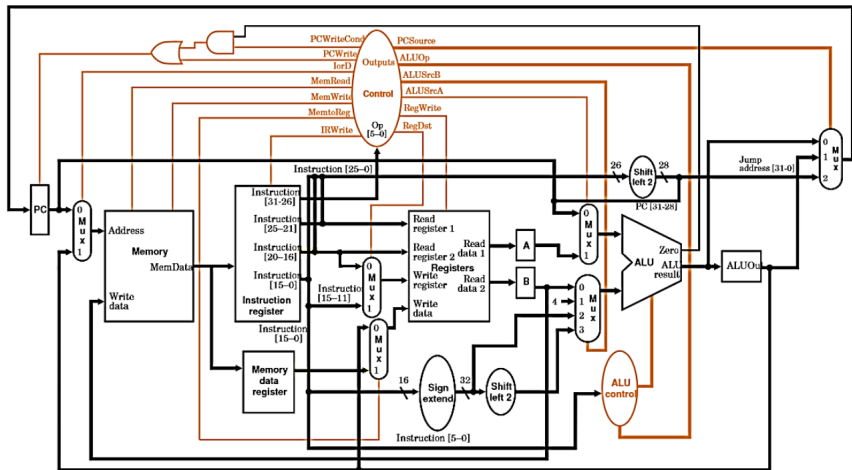
2025

- ▶ Controle Monociclo
- ▶ Controle Multiciclo
- ▶ Controle Pipeline
- ▶ Memória Cache

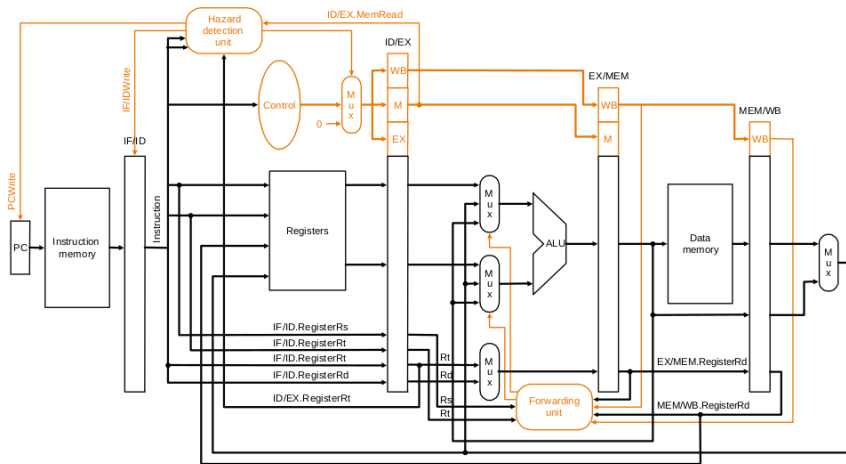
Controle Monociclo



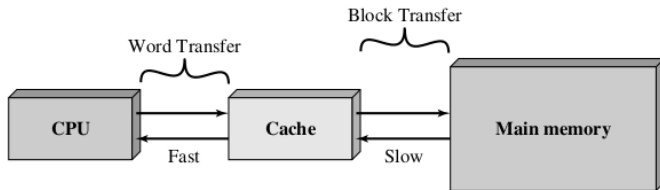
Controle Multiciclo



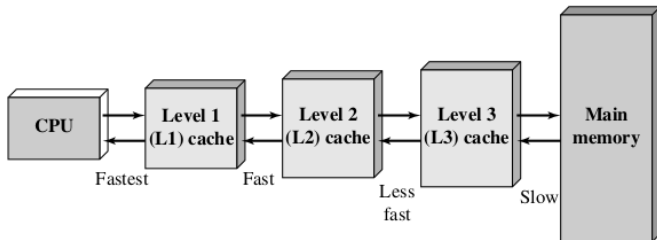
Control Pipeline



Memória Cache



(a) Single cache



(b) Three-level cache organization

Limitações

Monociclo:

- ▶ todas instruções pagam o mesmo preço

Multiciclo:

- ▶ não explora paralelismo

Pipeline:

- ▶ a vazão máxima (throughput) é limitada a uma instrução por ciclo
- ▶ a unificação de todos os tipos de instruções em um pipeline pode gerar projetos ineficientes
- ▶ se algum estágio não pode ser executado em um único ciclo, bolhas são induzidas em todo o pipeline
- ▶ como uniformizar instruções de memória, adição, multiplicação, ponto flutuante, etc.?

Controle Suprescalar

$$\text{Desempenho} = \# \text{instruções} \times \frac{\text{período de ciclo}}{\text{IPC}}$$

Estratégias¹:

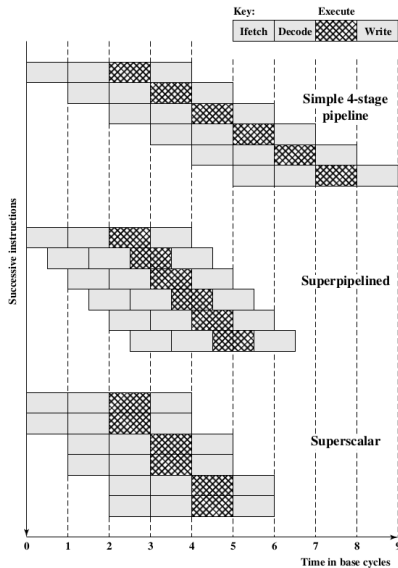
- ▶ instruções mais complexas, compilador otimizado
- ▶ pipeline mais profundo (mais estágios)
- ▶ pipeline mais largo (pipelines em paralelo)

Controle Suprescalar:

- ▶ paralelismo no nível de instruções e duplicação de recursos
- ▶ inicializa mais instruções por ciclo (pipelines paralelos)
- ▶ implementa pipelines heterogêneos (pipelines diversificados)
- ▶ permite execução de instruções fora de ordem (pipelines dinâmicos)

¹IPC = Instruções por Ciclo

Superscalar vs Superpipelined



Pipeline Paralelos

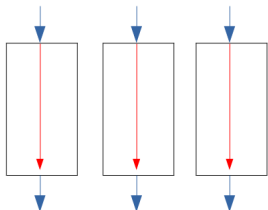
Sem Paralelismo



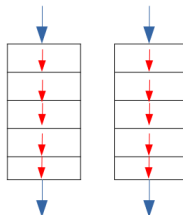
Paralelismo Temporal



Paralelismo Espacial

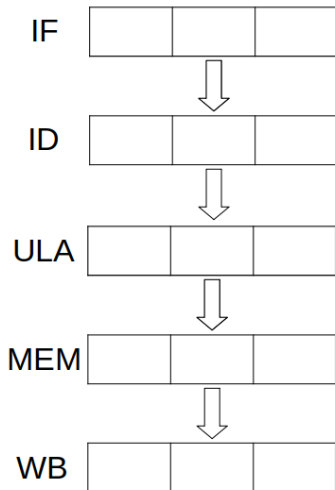


Pipeline Paralelo



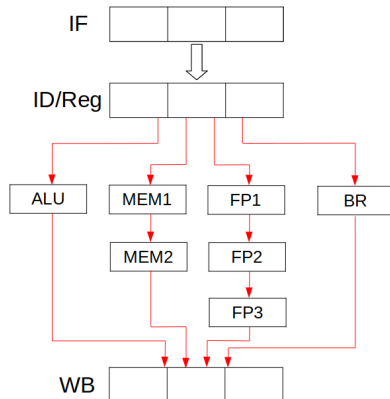
Pipeline Paralelos

- ▶ potencialmente, a cada ciclo, w instruções podem ser adicionadas ao pipeline
- ▶ hardware adicional para implementar pode aumentar em um fator de w
- ▶ registradores especiais entre estágios devem ser compartilhados entre os pipes paralelos (complexidade w^2)
- ▶ acesso concorrente aos registradores e às memórias caches de dados e instruções



Pipeline Diversificados

- ▶ projeto eficiente: cada pipe pode ser customizado para uma instrução particular
- ▶ se um tipo de instrução é frequente, esse tipo pode ser replicado
- ▶ cada instrução incorre apenas na latência necessária
- ▶ se dependências entre instruções são resolvidas anteriormente, os pipes podem ser controlados de forma distribuída e independente



Pipeline Rígido

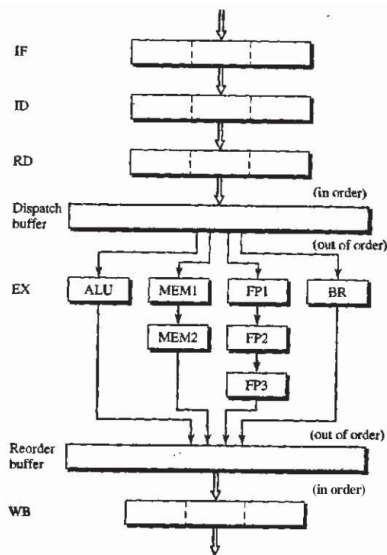
- ▶ se uma instrução é interrompida (por exemplo, cache miss), todas instruções posteriores também são interrompidas
- ▶ os registradores especiais em todos os pipes são interrompidos ou encaminhados em conjunto
- ▶ todas instruções entram e saem de cada estágio na mesma ordem que a sequência de código original
- ▶ em um pipeline paralelo, o custo dessa interrupção é multiplicado por w

Pipeline Dinâmico

- ▶ separar interrupções e encaminhamentos entre pipes
- ▶ necessidade de um controle unificado que controla os registradores especiais de cada pipe individualmente
- ▶ permitir que uma instrução posterior possa contornar/ultrapassar uma instrução interrompida
- ▶ mudança de ordem na execução das instruções
- ▶ instruções podem ser executadas assim que seus operandos estejam disponíveis

Pipeline Dinâmico

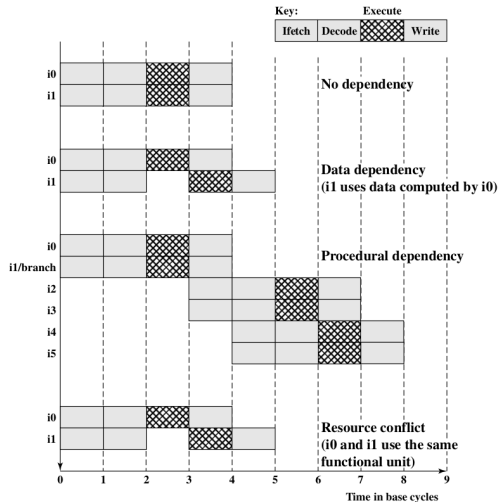
- ▶ buffers com multientradas: de despacho e reordenação
- ▶ buffer de despacho permite que as instruções cheguem e saem em ordem diferente
- ▶ instruções são despachadas se o recurso necessário (pipe) está disponível
- ▶ buffer de reordenação emite as instruções para o estágio de Write Back na ordem original



Limitações: conflitos

- ▶ dependência de desvio
- ▶ conflito de recurso
- ▶ dependência de dados verdadeira (read after write)
 - ▶ add \$r1,\$r1,\$r2
 - ▶ add \$r3,\$r1,\$r4
- ▶ dependência de saída (write after write)
- ▶ antidependência (write after read)

Limitações: conflitos



Política de Iniciação de Instruções

- ▶ Processador:
 - ▶ identifica o paralelismo no nível de instruções
 - ▶ coordena a busca, decodificação e execução de instruções em paralelo
- ▶ Iniciação de Instruções: processo de iniciar a execução de instruções nas unidades funcionais.
- ▶ Política: protocolo usado para iniciar a execução de instruções.

Política de Iniciação de Instruções

Tipos de Ordenação de Instruções

- ▶ Ordem na qual as instruções são buscadas
- ▶ Ordem na qual as instruções são executadas
- ▶ Ordem na qual as instruções atualizam o conteúdo de registradores e memória

Política de Iniciação de Instruções

- ▶ Iniciação em ordem com terminação em ordem
- ▶ Iniciação em ordem com terminação fora de ordem
- ▶ Iniciação fora de ordem com terminação fora de ordem

Exemplo

- ▶ instruções são buscadas duas por vez
- ▶ I1 precisa de dois ciclos para executar.
- ▶ I3 e I4 competem pelo mesmo recurso.
- ▶ I5 depende do valor produzido pelo I4.
- ▶ I5 e I6 competem pelo mesmo recurso.

Decode		Execute			Write	Cycle
I1	I2					1
I3	I4	I1	I2			2
I3	I4	I1				3
	I4			I3	I1 I2	4
I5	I6			I4		5
	I6		I5		I3 I4	6
			I6			7
					I5 I6	8

(a) In-order issue and in-order completion

Decode		Execute			Write	Cycle
I1	I2					1
I3	I4	I1	I2			2
	I4	I1		I3	I2	3
I5	I6			I4	I1 I3	4
	I6		I5		I4	5
			I6		I5	6
					I6	7

(b) In-order issue and out-of-order completion

Decode		Window	Execute			Write	Cycle
I1	I2						1
I3	I4	I1, I2	I1	I2			2
I5	I6	I3, I4	I1		I3	I2	3
		I4, I5, I6		I6	I4	I1 I3	4
		I5		I5		I4 I6	5
						I5	6

(c) Out-of-order issue and out-of-order completion

Iniciação em Ordem com Terminação Fora de Ordem

Dependência de Saída (escrita após escrita):

- ▶ $R3 := R3 + R5$; (I1)
- ▶ $R4 := R3 + 1$; (I2)
- ▶ $R3 := R5 + 1$; (I3)
- ▶ I2 depende do resultado de I1 – dependência de dados
- ▶ Se I3 termina antes de I1, o resultado de I1 estará errado - dependência de saída

Iniciação Fora de Ordem com Terminação Fora de Ordem

Antidependência (leitura após escrita)

- ▶ Com a iniciação fora de ordem, outro tipo de dependência pode ocorrer:
- ▶ $R3 := R3 + R5$; (I1)
- ▶ $R4 := R3 + 1$; (I2)
- ▶ $R3 := R5 + 1$; (I3)
- ▶ $R7 := R3 + R4$; (I4)
- ▶ I3 não pode terminar antes que I2 inicie já que I2 necessita de um valor em R3 e I3 modifica R3

Exercício

- ▶ A) Quais dependências existem no programa?
- ▶ B) Mostre a atividade do pipeline para o programa usando políticas:
 - ▶ Emissão em-ordem e conclusões em-ordem
 - ▶ Emissão em-ordem e conclusões fora-de-ordem
 - ▶ Emissão fora-de-ordem (considere uma janela de execução após o estágio de decodificação) e conclusões fora-de-ordem.

```
l1: lw $r1, 120($r0)
l2: add $r2,$r3,$r1
l3: add $r3,$r3,$r4
l4: mul $r4,$r4,$r5
l5: mul $r6,$r9,$r7
l6: and $r6,$r7,$r8
```

